

CCIS

كلية علوم الحاسب والمعلومات
COLLEGE OF COMPUTER &
INFORMATION SCIENCES



Flutter

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY



IS487 Emerging Topics in IS

Dr. Abbas Malik
amaalik@psu.edu.sa

Flutter Navigation

- Using Navigator
 - Push & Pop using Navigator
- Using Named Routes - Router
 - Create named routes, then push the named routes
 - Can be defined at the start of the app

Named Routes Navigation

- Simple and Easy to learn
- Built in Flutter Framework
- Declarative Route Definitions
- Simple stack management
- Built in back button handling
- Good for Simple Apps
- Deep links

Stack Management

```
// Push and replace  
Navigator.pushReplacementNamed(context, '/profile');  
  
// Pop until condition met  
Navigator.popUntil(context, ModalRoute.withName('/'));  
  
// Push and remove all until  
Navigator.pushNamedAndRemoveUntil(  
  context,  
  '/home',  
  (route) => false  
);
```

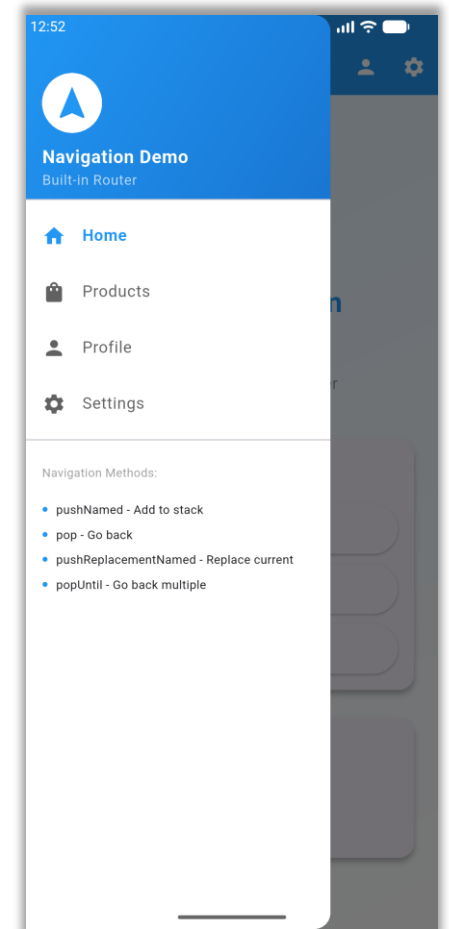
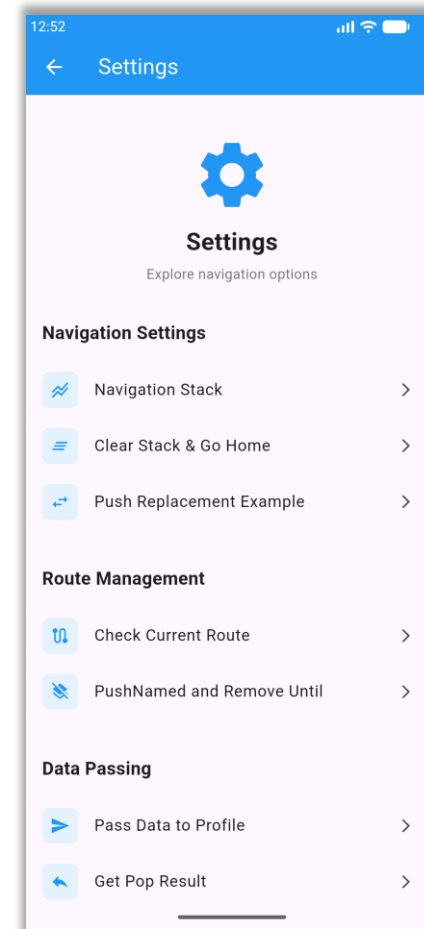
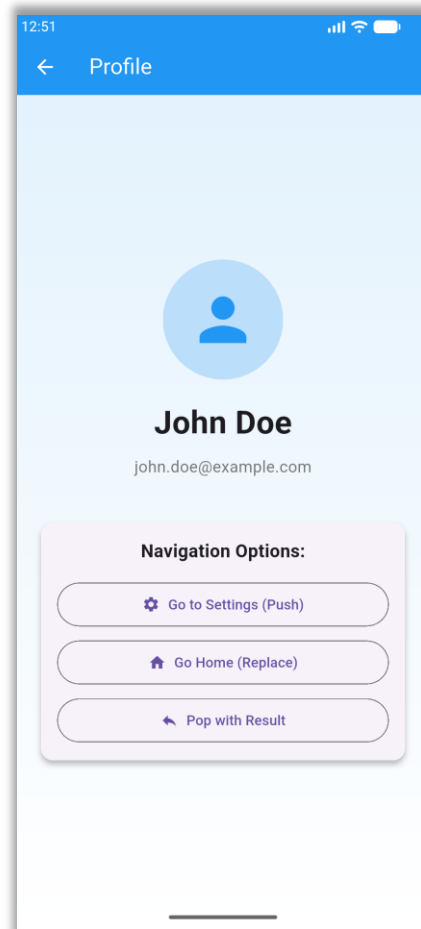
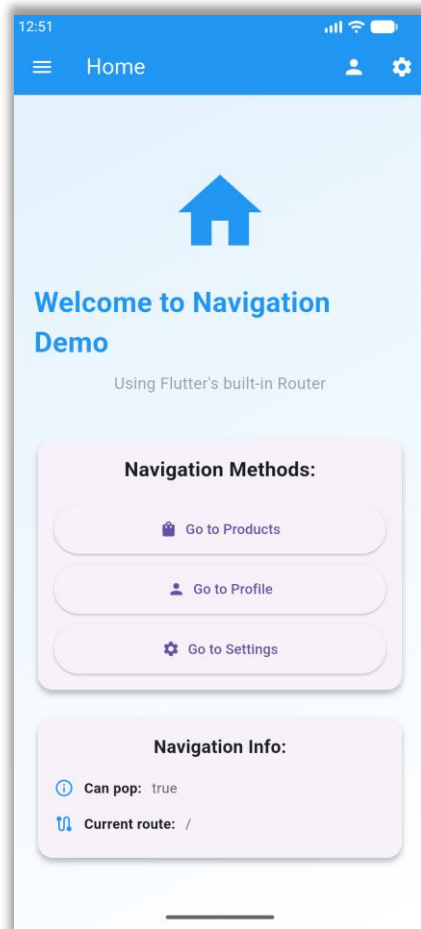
Data Passing

```
// Passing data  
Navigator.pushNamed(  
    context,  
    '/profile',  
    arguments: {'userId': 123, 'name': 'John'}  
);  
  
// Receiving data  
final args = ModalRoute.of(context)!.settings.arguments as Map;
```

Simple Return Values

```
// Page 1: Wait for result  
final result = await Navigator.pushNamed(context, '/profile');  
  
// Page 2: Return value  
Navigator.pop(context, 'Data from profile');
```

Sample App



Limitations

- Complex Navigation for large apps
- Apps needing advanced deep linking
- Web apps requiring URL synchronization
- Complex nested navigation
- Fine-grained navigation control

go_router Package

Feature	Named Routes	GoRouter
Setup Complexity	Simple	Moderate
Code Size	Small	Larger
Deep Linking	Basic	Advanced
URL Sync	No	Yes
Nested Routes	Manual	Built-in
Redirection	Manual	Built-in
Learning Curve	Easy	Steeper
Flexibility	Limited	High

Sensor/Hardware Integration

Sensors in Modern Devices

Motion Sensor

- Accelerometer: Detects acceleration, tilt, and shaking
- Gyroscope: Measures rotational speed and orientation: gaming and 360-degree photo viewing.
- Magnetometer: Functions as a compass to determine orientation relative to magnetic north.

Sensors in Modern Devices

Environmental Sensors

- Ambient Light Sensor: Adjusts screen brightness automatically to save battery.
- Proximity Sensor: Detects when the phone is held to the ear during a call, turning off the screen to prevent accidental touches.
- Barometer: Measures atmospheric pressure to calculate altitude for navigation or fitness tracking.

Sensors in Modern Devices

Position Sensor

- GPS Sensor: Tracks geographic coordinates, speed, and altitude for location-based services.

Sensors in Modern Devices

Biometric and Imaging Sensor

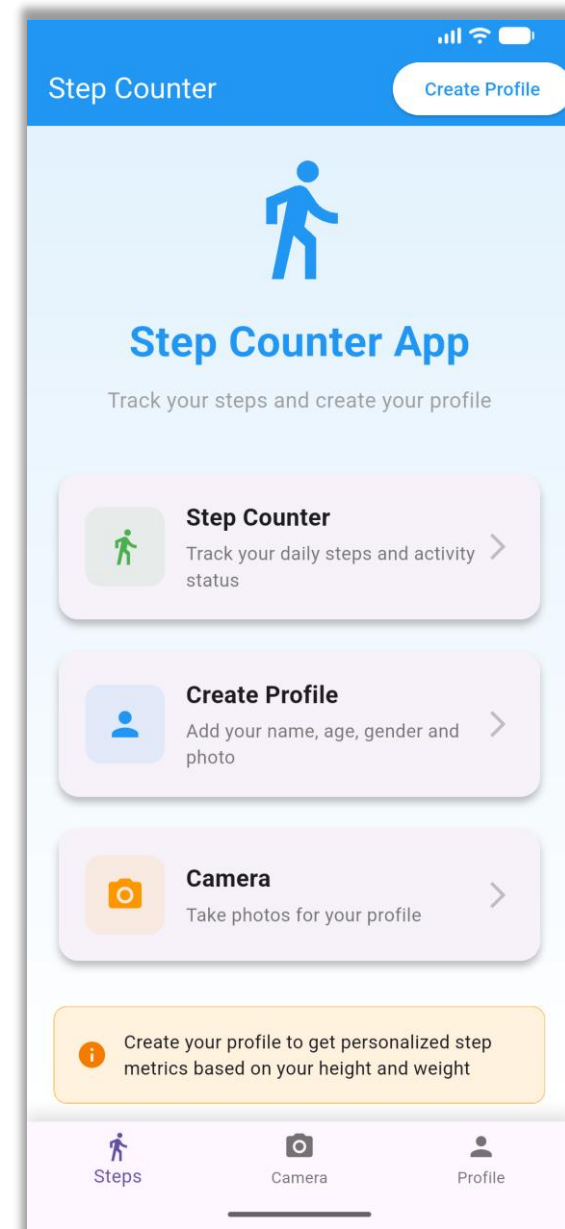
- Fingerprint Sensor: Provides secure authentication.
- Camera Sensor: Captures image and video data.
- Time-of-Flight (ToF) Sensor: Measures distance for portrait mode (bokeh effect) and faster autofocus.

Sensors in Modern Devices

Other Sensor

- NFC Sensor: Enables short-range communication for contactless payments.
- Thermometer/Thermistors: Monitors internal device temperature to prevent overheating.
- ...

Sensors App - Packages



Sensors App - Packages

Packages

- go_router
- flutter_riverpod
- Pedometer
- Camera
- image_picker
- path_provider
- path
- shared_preferences
- permission_handler

Sensors App - Permissions

AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA"/>  
<uses-permission android:name="android.permission.ACTIVITY_RECOGNITION"/>  
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

Sensors App - Models

UserProfile

```
class UserProfile {  
    final String name;  
    final String profileImagePath;  
    final int age;  
    final Gender gender;  
    final double? height; // in cm  
    final double? weight; // in kg  
    final DateTime? dateOfBirth;
```

```
enum Gender { male, female, other }  
  
extension GenderExtension on Gender {  
    String get displayName {  
        switch (this) {  
            case Gender.male:  
                return 'Male';  
            case Gender.female:  
                return 'Female';  
            case Gender.other:  
                return 'Other';  
        }  
    }  
}
```

Sensors App - Models

UserProfile

```
UserProfile copyWith({
    String? name,
    String? profileImagePath,
    int? age,
    Gender? gender,
    double? height,
    double? weight,
    DateTime? dateOfBirth,
}) {
    return UserProfile(
        name: name ?? this.name,
        profileImagePath: profileImagePath ?? this.profileImagePath,
        age: age ?? this.age,
        gender: gender ?? this.gender,
        height: height ?? this.height,
        weight: weight ?? this.weight,
        dateOfBirth: dateOfBirth ?? this.dateOfBirth,
    );
}
```

Sensors App - Models

UserProfile

```
Map<String, dynamic> toJson() {  
    return {  
        'name': name,  
        'profileImagePath': profileImagePath,  
        'age': age,  
        'gender': gender.index,  
        'height': height,  
        'weight': weight,  
        'dateOfBirth': dateOfBirth?.toIso8601String(),  
    };  
}
```

Sensors App - Models

UserProfile

```
factory UserProfile.fromJson(Map<String, dynamic> json) {  
  return UserProfile(  
    name: json['name'],  
    profileImagePath: json['profileImagePath'],  
    age: int.parse(json['age']),  
    gender: Gender.values[int.parse(json['gender'])],  
    height: double.parse(json['height']),  
    weight: double.parse(json['weight']),  
    dateOfBirth: json['dateOfBirth'] != null  
      ? DateTime.parse(json['dateOfBirth'])  
      : null,  
  );  
}
```

Sensors App - Models

UserProfile

```
// Calculate BMI
double? get bmi {
    if (height != null && weight != null) {
        final heightInMeters = height! / 100;
        return weight! / (heightInMeters * heightInMeters);
    }
    return null;
}

String get bmiCategory {
    final bmiValue = bmi;
    if (bmiValue == null) return 'Unknown';

    if (bmiValue < 18.5) return 'Underweight';
    if (bmiValue < 25) return 'Normal';
    if (bmiValue < 30) return 'Overweight';
    return 'Obese';
}
```


Sensors App - Models

CameraState

```
class CameraState {  
    final File? capturedImage;  
    final bool isCapturing;  
    final String? error;  
  
    CameraState({this.capturedImage, required this.isCapturing, this.error});  
  
    factory CameraState.initial() {  
        return CameraState(capturedImage: null, isCapturing: false, error: null);  
    }  
  
    CameraState copyWith({  
        File? capturedImage,  
        bool? isCapturing,  
        String? error,  
    }) {  
        return CameraState(  
            capturedImage: capturedImage ?? this.capturedImage,  
            isCapturing: isCapturing ?? this.isCapturing,  
            error: error,  
        );  
    }  
}
```

Sensors App - Models

StepData

```
class StepData {  
    final int stepCount;  
    final double distance; // in km  
    final double calories;  
    final DateTime timestamp;  
    final ActivityType activityType;  
}
```

Sensors App - Models

StepState

```
class StepState {  
    final int stepCount;  
    final double distance;  
    final double calories;  
    final ActivityType currentActivity;  
    final List<StepData> stepHistory;  
    final bool isMonitoring;  
    final String? error;  
}
```

Sensors App Providers (UserProfile)

```
class UserProfileNotifier extends AsyncNotifier<UserProfile> {
    @override
    FutureOr<UserProfile> build() {
        return _loadSavedUserProfile();
    }

    Future<UserProfile> _loadSavedUserProfile() async {
        // Load saved user profile
        final prefs = await SharedPreferences.getInstance();
        final savedProfile = prefs.getString('user_profile');
        if (savedProfile != null) {
            try {
                final Map<String, dynamic> json = Map.fromEntries(
                    savedProfile.split(',').map((e) {
                        final parts = e.split(':');
                        return MapEntry(parts[0], parts[1]);
                    }),
                ); // Map.fromEntries
                final userProfile = UserProfile.fromJson(json);
                return userProfile;
            } catch (e) {
                log('Error loading profile: $e');
            }
        }
        return UserProfile(
            name: '', age: 0, gender: Gender.other, profileImagePath: '',
        );
    }
}
```

Sensors App - Providers (**CameraState**)

```
class CameraNotifier extends AsyncNotifier<CameraState> {  
    final ImagePicker _picker = ImagePicker();  
  
    @override  
    FutureOr<CameraState> build() async {  
        return CameraState(isCapturing: false);  
    }  
}
```

Sensors App - Providers (**StepState**)

```
class StepNotifier extends AsyncNotifier<StepState> {
    UserProfile? userProfile;
    Stream<StepCount>? _stepCountStream;
    Stream<PedestrianStatus>? _pedestrianStatusStream;
    StreamSubscription<StepCount>? _stepSubscription;
    StreamSubscription<PedestrianStatus>? _statusSubscription;

    @override
    FutureOr<StepState> build() async {
        return StepState(
            stepCount: 0,
            distance: 0.0,
            calories: 0.0,
            currentActivity: ActivityType.stopped,
            stepHistory: [],
            isMonitoring: false,
            error: null,
        );
    }
}
```

```
Future<void> startMonitoring() async {  
  // state = const AsyncLoading();  
  if (state.value!.isMonitoring) return;  
  try {  
    _stepCountStream = Pedometer.stepCountStream;  
    _pedestrianStatusStream = Pedometer.pedestrianStatusStream;  
    _stepSubscription = _stepCountStream!.listen(  
      _onStepCount,  
      onError: _onStepError,  
    );  
    _statusSubscription = _pedestrianStatusStream!.listen(  
      _onStatusChange,  
      onError: _onStatusError,  
    );  
    state = AsyncData(state.value!.copyWith(isMonitoring: true, error: null));  
  } catch (error) {  
    state = AsyncData(  
      state.value!.copyWith(error: 'Error initializing pedometer: $error'),  
    );  
  }  
}
```

```
void _onStepCount(StepCount event) {  
    // state = const AsyncLoading();  
    final newStepCount = event.steps;  
    // Calculate metrics based on user profile if available  
    final newDistance = StepData.calculateDistance(  
        newStepCount,  
        userProfile?.height,  
    );  
    final newCalories = StepData.calculateCalories(  
        newStepCount,  
        userProfile?.weight,  
    );  
    state = AsyncData(  
        state.value!.copyWith(  
            stepCount: newStepCount,  
            distance: newDistance,  
            calories: newCalories,  
        ),  
    );  
}
```


Sensors App - Screens

- SplashScreen
- HomeScreen
- ProfileScreen
- CameraScreen
- StepCounterScreen

Sensors App Router

```
final appRouterProvider = Provider<GoRouter>((ref) {  
  return GoRouter(  
    initialLocation: '/',  
    debugLogDiagnostics: true,  
    routes: [  
      GoRoute(  
        path: '/',  
        name: 'splash',  
        pageBuilder: (context, state) {  
          return MaterialPage(child: const SplashScreen());  
        },  
      ), // GoRoute  
      ShellRoute(  
        builder: (context, state, child) {  
          return HomeScreen(child: child);  
        },  
        routes: [  
          GoRoute(  
            path: '/home',  
            name: 'home',  
            pageBuilder: (context, state) {  
              return const NoTransitionPage(child: HomeContent());  
            },  
          ), // GoRoute  
        ],  
      ),  
    ],  
  );  
});
```

App

```
class StepCounterApp extends ConsumerWidget {
  const StepCounterApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(appRouterProvider);

    return MaterialApp.router(
      title: 'Step Counter App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: true,
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
          elevation: 0,
        ), // AppBarTheme
      ), // ThemeData
      routerConfig: router,
      debugShowCheckedModeBanner: false,
    ); // MaterialApp.router
  }
}
```

Main

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  runApp(ProviderScope(child: const StepCounterApp()));  
}
```

References

- <https://docs.flutter.dev/ui/navigation>
- <https://firebase.google.com/docs/guides>